

雲端資料庫: Firebase

冷資料: 較少用的資料(照片) , 流通比較慢的資料 , firestore database

熱資料: 大量普遍用的資料 , 流通比較快的資料 , realtime database

電腦第一天: 1970/01/01(以此為0)

timestamp(時間戳記): 現在時間 - 電腦第一天

SQL: 關聯式資料庫 , 用於結構化資料

NoSQL: 非關聯式資料庫 , 用於半結構、非結構化資料 , 不需要設定每個欄位需要什麼內容 , 即不需先設定資料庫架構就能直接使用 , 如json

firebase: NoSQL , 不用訂定型態 , 但是值還是有對應型態

firebase設定: Data弄好 , 要按新增。在欄位修改資料後按Enter

根: "https://djangotest-f3d94-default-rtdb.firebaseio.com/"

產生節點(child): database.child("Data").child("Morty")

Push: 沒有結點名稱 , 而是亂數

data = {"name": "Mortimer 'Morty' Smith"}

database.child("Data").push(data)

設置:

data = {"name": "Mortimer 'Morty' Smith"}

database.child("Data").child("Morty").set(data)

更新: database.child("Data").child("Morty").update({"name": "Mortiest Morty"})

刪除: database.child("Data").child("Morty").remove()

多位子更新: 不僅更新 , 也能直接設置新值

```
data = {  
  "Data/Morty/": {"name": "Mortimer 'Morty' Smith"},  
  "Data/Rick/": {"name": "Rick Sanchez"}  
}
```

database.update(data)

亂數key: "Data/" + database.generate_key(): {"name": "Mortimer 'Morty' Smith"},

印出: 顯示在終端機。 .each()子節點

all_Data = database.child("Data").get()

for data in all_Data.each():

print(data.key())

print(data.val())

驗證: 建構 -> Authentication

Token: 過期 , 曾經登入過

Firebase 專案總覽 -> 專案設定 -> 一般設定最下面刪除專案

線性迴歸不能拿來做分類法 , 如生死(二元)歸類問題

解釋能力: 若非簡單迴歸 , 如好幾十個欄位 , 則無法計算 , 要用梯度法不斷迭代

梯度下降法: 梯度為負 , 故找最小值。非凸函數或陷入局部最小值找不到全域最小值

梯度上升法: 梯度為正 , 故找最大值

損失函數要找的是最少誤差 , 所以採用梯度下降法找損失函數(誤差)的最小值

凸函數: 任意兩點連成的線段 , 皆位於圖形的上方的實值函數

假設有一個簡單的線性回歸模型 $y = w_1x + w_0$, 目標是找到最優權重 $w = [w_0, w_1]^T$ 以最小化均方誤差。

1. 計算梯度

損失函數：

$$\mathcal{L}(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n (y_i - (w_1 x_i + w_0))^2$$

對權重 w_0 和 w_1 求偏導：

$$\frac{\partial \mathcal{L}}{\partial w_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - (w_1 x_i + w_0))$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - (w_1 x_i + w_0))$$

fetch下載的資料 -> 存在user底下的資料夾

每張照片都去問別人是不是這個人 , 所以雖然是二元分類 , 但卻有7個結果

分類法: 不是得出數字 , 而是分到各類別的機率

邏輯斯迴歸: 二元分類法 , 把數值丟進S函數(sigmoid、logistic)進行轉換成機率值範圍[0, 1] , 再依機率進行分類 , 即輸出分類結果(0或1) 。機率相加為1。後面多元分類法機率相加不為1。

線性迴歸產生線上x跟y的對應關係 , 分類法產生分類(預測)線切割(分)出上下兩區域(或大於兩區域) 。根據梯度上升(下降)法找特徵的權重值

預測結果要包含: 1.分類報告

Precision (0) = 0.82 → 被預測為「未生還 (0)」的人中 , 82% 真的沒生還。

Precision (1) = 0.76 → 被預測為「生還 (1)」的人中 , 76% 真的生還。

Accuracy = 0.80 → 80% 的乘客預測正確。

F1-score → 精確率和召回率的綜合指標。

Recall (0) = 0.84 → 所有真實是 0 的樣本中 , 84%被成功預測為 0 , 16%被遺漏。

Recall (1) = 0.73 → 所有真實是 0 的樣本中 , 73%被成功預測為 0 , 27%被遺漏。

類別 0 (陰性) 預測較準確 , 召回率達到 84% , 但對 類別 1 (陽性) 的召回率 只有 73% , 可能會漏掉一些實際為 1 的案例。

選擇高 Precision , 確保預測為正的案例「真的正確」 ,

選擇高 Recall , 確保真正的正例「不會被漏掉」 ,

依損失代價高來選擇 , 如果兩者都很重要就使用 F1-score

C值: 懲罰 , 超參數之一 , 正則化強度的倒數 , 越低越正規化 , 模型越簡單 , 更不擬合(涵蓋)數據。以訓練集與測試集的準確率差異來判斷擬合程度 , 趨於0則越擬合 ,

在過擬合不大的情況下 , 還是以測試準確率為優先。

logreg = LogisticRegression(C=1).fit(X_train, y_train)

特徵標準化: Z分數 , $X' = \frac{X-\mu}{\sigma}$ 。資料處理前的準備(起手式)。提高準確度 , 每筆資料去除單位的影響性 , 單位的影響性會造成計算上的誤差。例如: 年薪影響特徵有 , 年齡

(15 , 20 , 25)、收入(50000 , 55000 , 60000) ; 標準化後兩者都變成(-1 , 0 , 1)

多元分類法:

1.中位數: Tree Based -> Decision Tree(決策樹) , Random Forest , Xgboost

2.平均數: Distanced Based -> Nearest Neighbors , Support Vector Machines (SVM)

決策樹: 以查表方式 , 用看的 , 不用計算。速度快但準確度差 , 容易分支過多 , 過度擬合。數字型資料須轉成類別型(套件自動) , 如0~100歲 , 0~20區間為一類別 , 以此類推

甜蜜點(Sweet spot): 模型複雜度中庸 , 準確率達最高點 ; 反之低或高 , 準確性都不高。

模型有誤差是好事 , 準確率百分百不一定好 , 會過擬合

預先修剪: 設定樹的深度 , 長到第幾層

後期修剪: 先長好再裁剪 , 套件不支援

Ensemble Methods集成方法: 結合多個模型來增加準確度

1.各個分類器之間須具有差異性

2. 每個分類器的準確度必須大於0.5

CART: Classification And Regression Trees , 綜合分類樹和迴歸樹。須先將訓練樣本進行分化 , 產生多棵具差異性的CART , 再將多棵差異性的樹進行Ensemble Method。

1.Bagging(資料重複取樣 , 取後放回): 不斷考試 , 每次都是強分類器。

數值 , 每次結果取平均(MSE概念) ; 分類 , 投票(分類)法

2.Boosting: 針對錯誤部份加強學習以提昇整體的效率 , 重點是將大量弱學習的分類器

(效率比較沒那麼好)逐步訓練成一個較強的分類器。速度較慢。深度學習的前身。

XGBoost , 機器學習中最強演算法

CART的屬性選擇指標: 吉尼係數

Random Forests: 速度慢。先針對資料(列)進行Bagging抽樣生成不同的訓練子集 , 然後在每個子集上建構CART , 並且樹的每個節點分裂時會對特徵(欄)進行抽樣 , 最後形成隨機森林

隨機森林是Bagging的一個應用(增加了特徵抽樣) , Bagging還能應用於其他模型上

機器學習都不會用到GPU

前面皆為觀察型演算法。

KNN: 隨便抓五人 , 有三人藍色兩人綠色 , 那就是藍色。

給定訓練資料建立模型 , 每個點抓一個最近的鄰居(K=1)。預測目標落入哪個資料點的鄰近距離就會被歸屬該類別。若K=5 , 考慮的鄰居較多 , 可能導致劃分區域有空白(無法判別) 。計算型演算法。類別資料須轉數值資料並標準化。K 的上限為訓練樣本數的20%。

假設K在8~12之間準確率相同 , 會挑中間K=10。挑少速度快 , 挑多符合多情況(擬合低)。

最大的優點為最慢的演算法 , 可做很多改良 , 如距離近權重高 , 遠則權重低。

例子: 1張100%中的圖 , 3張30%綸的圖 , KNN會判定像綸(局外人參與)。

但如果1張100%中的圖 , 3張80%綸的圖 , KNN會判定像綸(核心人士參與)。

把所有訓練資料儲存 , 做預測時才進行計算。預測點會跟所有訓練資料點計算距離